

Sentinel-AI: An End-to-End Enterprise Multimodal Financial Risk & Simulation Platform

Final Year Project Report — Design, Architecture & Implementation
Blueprint

Abderrahemane Sahki (Abdou)

ESTIN Béjaïa — AI & Data Science Engineering, Academic Year 2025/2026

Contents

1 Abstract	3
2 Chapter 1: Introduction	4
2.1 1.1 Context and Motivation	4
2.2 1.2 Problem Statement	4
2.3 1.3 Objectives	4
2.4 1.4 Scope and Limitations	5
2.5 1.5 Report Structure	5
3 Chapter 2: Background and Related Work	6
3.1 2.1 Tabular Risk Modeling	6
3.2 2.2 Multimodal Learning in Finance	6
3.3 2.3 Retrieval-Augmented Generation for Grounded Explanation	6
3.4 2.4 Reinforcement Learning for Portfolio Management	6
3.5 2.5 Explainability, Robustness, and Fairness	6
3.6 2.6 MLOps and Reproducible ML Systems	7
4 Chapter 3: Requirements Analysis	8
4.1 3.1 Functional Requirements	8
4.2 3.2 Non-Functional Requirements	8
4.3 3.3 Primary Use Cases	9
5 Chapter 4: System Architecture	10
5.1 4.1 Architectural Style	10
5.2 4.2 High-Level Component Diagram	10
5.3 4.3 Monorepo Layout	11
5.4 4.4 Technology Stack Rationale	11
5.5 4.5 Data Flow Summary	12
6 Chapter 5: Implementation — The Five-Phase Roadmap	13
6.1 5.1 Phase 1 — Core Engine (Tabular Risk Scoring)	13

6.2	5.2 Phase 2 — Multimodal Engine (Text + Vision Fusion)	14
6.3	5.3 Phase 3 — RAG + Grounded LLM Explanation	16
6.4	5.4 Phase 4 — Reinforcement Learning Agent	17
6.5	5.5 Phase 5 — Trust, Fairness, and MLOps Maturity	19
7	Chapter 6: Evaluation Methodology	21
8	Chapter 7: Testing Strategy	21
9	Chapter 8: Ethical, Safety, and Compliance Considerations	22
10	Chapter 9: Risk Register (Engineering Delivery Risks)	22
11	Chapter 10: Future Work	23
12	Chapter 11: Conclusion	24
13	References	24
14	Appendix A: Glossary	24

1 Abstract

Sentinel-AI is a modular, enterprise-grade platform for **financial risk research and simulation**, designed to consolidate the full spectrum of modern AI engineering — data pipelines, classical machine learning, computer vision, natural language processing, retrieval-augmented generation (RAG), reinforcement learning (RL), explainability, and MLOps — into a single, coherently engineered monorepo. Unlike a conventional trading or advisory system, Sentinel-AI is explicitly and irrevocably scoped as a **research and simulation platform**: it never issues real financial advice and never executes real-world trades. Its purpose is pedagogical and architectural — to serve as a rigorous, production-shaped sandbox in which every major discipline of the AI Mastery Roadmap is implemented against a single, realistic, high-stakes domain: financial risk.

This report documents the complete system design of Sentinel-AI: its problem framing, requirements, microservice architecture, technology stack, and the five-phase implementation roadmap that carries the system from a tabular gradient-boosting risk engine (Phase 1) through multimodal fusion of news and chart data (Phase 2), retrieval-grounded LLM explanations (Phase 3), a reinforcement-learning portfolio simulator (Phase 4), to a final layer of trust, fairness, and operational maturity (Phase 5). For each phase, the report presents the relevant mathematical formulation, the service-level design, representative implementation code, and the evaluation methodology used to validate that phase before promotion to the next.

The report closes with a discussion of ethical and regulatory guardrails, a risk register for the engineering effort itself, and a roadmap for future extensions beyond the initial five phases.

2 Chapter 1: Introduction

2.1 1.1 Context and Motivation

Modern quantitative finance sits at the intersection of several AI subfields that are rarely taught, or built, together: tabular modeling for market signals, NLP for unstructured text (news, filings), computer vision for chart-pattern recognition, sequential decision-making for portfolio allocation, and — increasingly — large language models used not as oracles but as *grounded explainers* of what a system has already decided. Most educational or portfolio projects isolate one of these disciplines. Sentinel-AI is motivated by the opposite premise: that mastering AI engineering end-to-end requires forcing all of these disciplines to interoperate inside one architecture, under one set of production constraints (latency, reproducibility, observability, fairness, and safety).

Financial risk is chosen as the domain not because the platform aims to compete with real trading infrastructure, but because it is one of the few domains rich enough to legitimately require every layer of the roadmap: structured time series, unstructured text, visual chart patterns, sequential decisions under uncertainty, and a hard requirement for explainability and auditability.

2.2 1.2 Problem Statement

There is no single, disciplined reference implementation that demonstrates how classical ML, computer vision, NLP, RAG-grounded LLMs, reinforcement learning, and MLOps trust tooling can be composed into one coherent, decoupled system rather than five disconnected notebooks. Sentinel-AI addresses this gap by defining:

1. A **monorepo architecture** that keeps each discipline as an independently deployable microservice while sharing common infrastructure (feature store, model registry, event bus).
2. A **five-phase delivery plan** that mirrors how a real AI platform team would incrementally de-risk and ship such a system.
3. **Hard safety boundaries** — the system reasons about risk and explains outcomes, but never acts as a financial advisor and never touches real capital.

2.3 1.3 Objectives

- Design and implement a decoupled, multi-service architecture for financial risk simulation using FastAPI, PyTorch, Docker, Kubernetes, MLflow, Redis, and PostgreSQL.
- Build a tabular risk-scoring engine (gradient boosting on engineered/PCA-reduced features) as the system’s core, with full experiment tracking.
- Extend the core engine with multimodal signals: FinBERT-based news-sentiment embeddings and CNN-based candlestick/chart-pattern recognition, fused into a unified risk representation.
- Implement a retrieval-augmented generation layer that grounds natural-language explanations of model outputs in retrieved SEC filings, financial news,

and economic reports — without allowing the LLM to generate independent market predictions.

- Implement a simulated market environment and train reinforcement-learning agents (Q-Learning → PPO) for risk-adjusted, simulation-only portfolio management.
- Instrument the entire system with explainability (SHAP/LIME), adversarial robustness testing, differential privacy experiments, subgroup bias auditing, drift detection, and CI/CD-driven Kubernetes deployment.

2.4 1.4 Scope and Limitations

In scope: synthetic and historical market data, simulated trading environments, research-grade model training and evaluation, RAG-grounded explanation generation, and the full MLOps lifecycle around these components.

Explicitly out of scope: live brokerage integration, real order execution, personalized financial advice, and any regulatory-grade investment recommendation. These exclusions are treated as architectural constraints, not just disclaimers — for example, the API gateway enforces that no endpoint can accept live account credentials or route to a broker, and the LLM service is contractually restricted (via prompt-and-tool design) to explanation-only outputs conditioned on retrieved documents and precomputed model outputs.

2.5 1.5 Report Structure

Chapter 2 surveys the technical background for each subsystem. Chapter 3 defines functional and non-functional requirements. Chapter 4 presents the system architecture. Chapter 5 walks through the implementation of all five phases with formulas and code. Chapter 6 defines the evaluation methodology. Chapter 7 covers the testing strategy. Chapter 8 addresses ethics and compliance. Chapter 9 presents a risk register. Chapter 10 discusses future work, and Chapter 11 concludes.

3 Chapter 2: Background and Related Work

3.1 2.1 Tabular Risk Modeling

Classical financial risk scoring relies on engineered features (returns, realized volatility, technical indicators, liquidity measures) reduced via dimensionality-reduction techniques such as Principal Component Analysis (PCA), and modeled with tree ensembles — most notably Gradient Boosting Machines (GBM), which remain a strong, interpretable baseline against deep tabular models for structured financial data due to their robustness to heterogeneous feature scales and missing data.

3.2 2.2 Multimodal Learning in Finance

Two unstructured modalities are especially informative for risk: **text** (news, filings, analyst commentary) and **images** (candlestick charts, technical-pattern visualizations). Transformer-based language models fine-tuned on financial corpora (e.g., FinBERT) produce sentiment and stance embeddings from text, while convolutional neural networks (CNNs) trained on rendered chart images can recognize recurring visual patterns (head-and-shoulders, double tops, flags) that traders use heuristically. Fusing these modalities with the tabular signal typically outperforms any single modality in predictive tasks that involve regime shifts or sentiment-driven volatility.

3.3 2.3 Retrieval-Augmented Generation for Grounded Explanation

LLMs are prone to hallucination when asked to reason about numeric or causal claims from memory. RAG mitigates this by retrieving relevant, verifiable source documents (SEC filings, news articles, economic reports) from a vector database and conditioning generation on that retrieved context. In Sentinel-AI, this pattern is deliberately constrained: the LLM is a **post-hoc explainer**, never a predictor — it receives the risk engine’s already-computed outputs plus retrieved evidence, and produces a natural-language explanation grounded in both.

3.4 2.4 Reinforcement Learning for Portfolio Management

Portfolio allocation is naturally framed as a sequential decision problem under uncertainty: an agent observes market state, selects an allocation (action), and receives a risk-adjusted return (reward) over time. Simple tabular Q-Learning is a useful pedagogical starting point on discretized state/action spaces, but does not scale to continuous, high-dimensional financial state representations. Proximal Policy Optimization (PPO), a policy-gradient method with a clipped surrogate objective, is the standard choice for continuous control problems requiring training stability, and is the target algorithm for Sentinel-AI’s mature RL agent.

3.5 2.5 Explainability, Robustness, and Fairness

Model trust in a risk-sensitive domain requires more than accuracy. SHAP (Shapley Additive Explanations) and LIME (Local Interpretable Model-Agnostic Explanations)

provide local and global attributions of model predictions. Adversarial robustness testing evaluates sensitivity to small, worst-case input perturbations. Differential privacy bounds what an attacker can infer about individual training records. Subgroup bias auditing checks that risk scores do not systematically disadvantage protected or sensitive segments of the (synthetic) population. Drift detection monitors whether the live data distribution has diverged from the training distribution, triggering retraining.

3.6 2.6 MLOps and Reproducible ML Systems

MLflow provides experiment tracking, model versioning, and a model registry. Docker and Kubernetes provide reproducible, scalable deployment units. Redis serves as a low-latency cache and message broker between services; PostgreSQL serves as the system of record for structured metadata, user/session state, and audit logs. Together these form the operational backbone that turns a collection of trained models into a maintainable platform.

4 Chapter 3: Requirements Analysis

4.1 3.1 Functional Requirements

ID	Requirement
FR-1	The system shall ingest historical and synthetic market data (OHLCV) and financial news into a unified data layer.
FR-2	The system shall compute engineered features and a PCA-reduced representation, then produce a GBM-based risk/volatility score per asset per time step.
FR-3	The system shall compute a news-sentiment embedding (FinBERT) and a chart-pattern embedding (CNN) and fuse them with the tabular signal into a unified multimodal risk representation.
FR-4	The system shall retrieve relevant SEC filings, news, and economic reports from a vector database and generate a grounded natural-language explanation of any risk score on request.
FR-5	The system shall simulate a market environment in which an RL agent selects portfolio allocations and receives risk-adjusted rewards, without any live trade execution.
FR-6	The system shall provide SHAP/LIME explanations, drift alerts, and subgroup bias reports for every deployed model version.
FR-7	The system shall track every experiment, model version, and deployment through MLflow, with full lineage from raw data to serving.
FR-8	The system shall expose all functionality through a versioned API Gateway and a monitoring dashboard, and shall refuse any request that implies live trading or personalized financial advice.

4.2 3.2 Non-Functional Requirements

- **Modularity:** each ML discipline is an independently deployable service with its own Dockerfile, dependencies, and MLflow experiment namespace.
- **Reproducibility:** every trained model is traceable to a specific dataset version, code commit, and hyperparameter set via MLflow.
- **Observability:** all services emit structured logs and Prometheus-compatible metrics; drift and bias monitors run on a schedule.
- **Safety:** the LLM service has no tool access to execution systems; the API gateway enforces a hard allow-list of endpoint categories (simulation, explanation, monitoring — never execution).
- **Scalability:** stateless services scale horizontally under Kubernetes; Redis absorbs bursty read load; heavy training jobs run as batch Kubernetes Jobs, not inside request-serving pods.
- **Testability:** every service maintains unit tests for its core logic and integration tests for its API contract.

4.3 3.3 Primary Use Cases

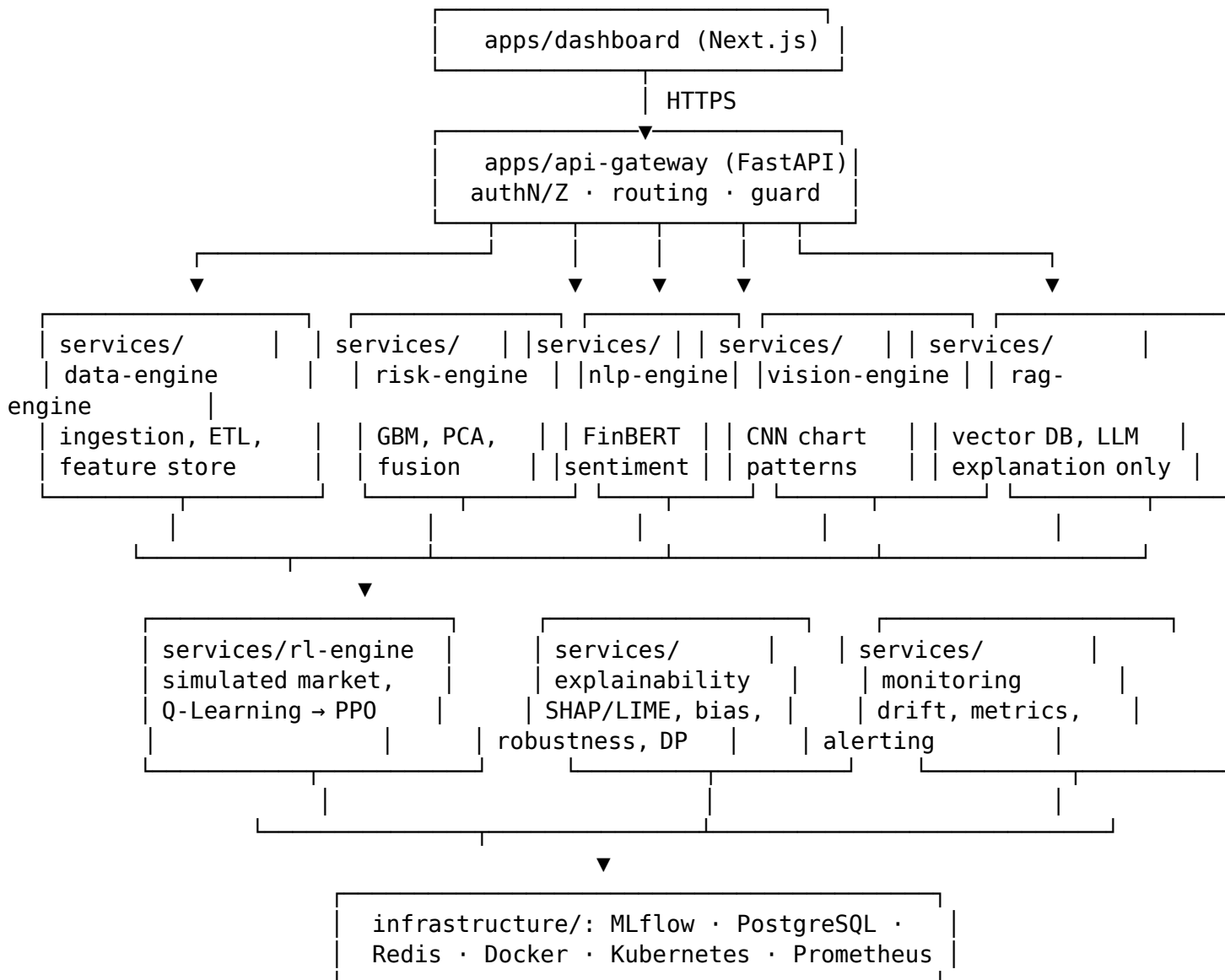
1. A researcher requests a risk score for a synthetic asset over a historical window and receives a numeric score plus a RAG-grounded explanation.
2. A researcher runs a simulated portfolio-management episode and inspects the RL agent's allocation decisions and cumulative risk-adjusted reward.
3. An MLOps engineer inspects a model card showing SHAP attributions, subgroup fairness metrics, and current drift status before promoting a model from staging to production in the registry.

5 Chapter 4: System Architecture

5.1 4.1 Architectural Style

Sentinel-AI follows a **modular monorepo with decoupled microservices**: a single Git repository hosts independently buildable and deployable services, sharing common tooling (linting, CI, infrastructure-as-code) but never sharing runtime state directly — all cross-service communication happens through well-defined REST/gRPC APIs, an event bus (Redis Streams), or shared storage layers (PostgreSQL, the feature store, the MLflow registry). This avoids both extremes: a monolithic notebook codebase (unmaintainable, untestable) and a fully polyrepo microservice sprawl (excessive operational overhead for a project of this scope).

5.2 4.2 High-Level Component Diagram



5.3 4.3 Monorepo Layout

```

sentinel-ai/
├── apps/
│   ├── dashboard/           # Next.js frontend
│   └── api-gateway/         # FastAPI/NestJS gateway, authN/Z, request routing
├── services/
│   ├── data-engine/         # Ingestion, cleaning, feature store
│   ├── risk-engine/         # PCA, GBM, volatility scoring, fusion head
│   ├── nlp-engine/          # FinBERT sentiment/embedding service
│   ├── vision-engine/       # CNN chart-pattern recognition service
│   ├── rag-engine/          # Vector DB, retriever, grounded LLM explainer
│   ├── rl-engine/           # Simulated market env, Q-Learning/PP0 agents
│   ├── explainability/      # SHAP/LIME, adversarial tests, DP, bias audits
│   └── monitoring/          # Drift detection, metrics, alerting
├── ml/
│   └── datasets/             # Versioned raw/processed data references (DVC-
style)
│   ├── experiments/         # MLflow experiment configs
│   ├── models/              # Serialized model artifacts (registry-linked)
│   ├── training/            # Training scripts per phase
│   └── evaluation/          # Evaluation & reporting scripts
├── infrastructure/
│   ├── docker/              # Per-service Dockerfiles, docker-compose
│   ├── kubernetes/          # Manifests / Helm charts
│   ├── mlflow/              # Tracking server + registry config
│   └── monitoring/          # Prometheus/Grafana configs
├── pipelines/
│   ├── ingestion/           # Scheduled data ingestion DAGs
│   ├── training/            # Training pipeline DAGs
│   └── inference/           # Batch/real-time inference DAGs

```

5.4 4.4 Technology Stack Rationale

Layer	Technology	Rationale
API Gateway	FastAPI (+ optional NestJS for the dashboard's BFF)	Async-first, typed, auto-documented (OpenAPI), aligns with existing Python ML stack
Model Training/Serving	PyTorch	Unified framework across NLP, CV, and RL services
Classical ML	scikit-learn / XGBoost / LightGBM	Mature, fast GBM implementations with strong tabular performance

Layer	Technology	Rationale
Experiment Tracking	MLflow	Tracking, model registry, and reproducibility in one tool
Cache / Event Bus	Redis	Sub-millisecond feature/prediction caching and lightweight pub/sub between services
System of Record	PostgreSQL	ACID-compliant storage for metadata, audit logs, session state
Vector Store	pgvector (on PostgreSQL) or a dedicated vector DB (e.g., Qdrant)	Keeps infra minimal in early phases; can be swapped for a dedicated store as retrieval scale grows
Containerization	Docker	Reproducible, per-service build artifacts
Orchestration	Kubernetes	Horizontal scaling, rolling deploys, resource isolation per service
CI/CD	GitHub Actions	Native integration with the monorepo, matrix builds per service

5.5 4.5 Data Flow Summary

1. **Ingestion:** data-engine pulls historical/synthetic OHLCV data and financial news/filings, writes cleaned data to the feature store (PostgreSQL) and raw documents to object storage for later vectorization.
2. **Feature computation:** engineered features are computed and cached in Redis for low-latency scoring; a PCA transform (fit during training, versioned in MLflow) reduces dimensionality.
3. **Scoring:** risk-engine combines the PCA-reduced tabular signal with embeddings from nlp-engine and vision-engine to produce a unified risk score.
4. **Explanation:** on request, rag-engine retrieves supporting documents and generates a grounded explanation of the score — never a new prediction.
5. **Simulation:** rl-engine consumes the risk engine’s outputs as part of its state representation and simulates portfolio decisions inside a closed market environment.
6. **Trust layer:** explainability and monitoring continuously audit every deployed model for attribution quality, robustness, fairness, and drift, feeding results back into MLflow and the dashboard.

6 Chapter 5: Implementation — The Five-Phase Roadmap

Each phase below is designed to be independently demonstrable and independently testable before the next phase begins, mirroring how a real ML platform team de-risks delivery.

6.1 5.1 Phase 1 — Core Engine (Tabular Risk Scoring)

Goal: establish the data pipeline, feature engineering, dimensionality reduction, and a gradient-boosting risk/volatility scorer, fully tracked in MLflow.

Feature engineering. For an asset price series P_t , the log return is $r_t = \ln(P_t/P_{t-1})$. Realized volatility over a rolling window of size w is estimated as:

$$\hat{\sigma}_t = \sqrt{\frac{1}{w-1} \sum_{i=0}^{w-1} (r_{t-i} - \bar{r})^2}$$

Additional engineered features include rolling momentum, RSI, moving-average crossovers, and liquidity proxies (volume z-scores).

Dimensionality reduction (PCA). Given a standardized feature matrix $X \in \mathbb{R}^{n \times d}$, PCA finds the projection $Z = XW$ where W contains the top- k eigenvectors of the covariance matrix $\Sigma = \frac{1}{n-1} X^\top X$, retaining components that explain a target cumulative variance (e.g., 95%).

```
# ml/training/phase1_feature_pipeline.py
```

```
import pandas as pd
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from dataclasses import dataclass
```

```
@dataclass
```

```
class FeatureConfig:
```

```
    rolling_window: int = 20
```

```
    variance_target: float = 0.95
```

```
def engineer_features(df: pd.DataFrame, cfg: FeatureConfig) -> pd.DataFrame:
```

```
    out = df.copy()
```

```
    out["log_return"] = np.log(out["close"] / out["close"].shift(1))
```

```
    out["volatility"] = out["log_return"].rolling(cfg.rolling_window).std()
```

```
    out["momentum"] = out["close"].pct_change(cfg.rolling_window)
```

```
    out["volume_z"] = (
```

```
        out["volume"] - out["volume"].rolling(cfg.rolling_window).mean()
```

```
) / out["volume"].rolling(cfg.rolling_window).std()
```

```
    return out.dropna()
```

```
def reduce_dimensions(X: np.ndarray, cfg: FeatureConfig) -> tuple[np.ndarray, PCA, Stan
```

```

scaler = StandardScaler().fit(X)
X_scaled = scaler.transform(X)
pca = PCA(n_components=cfg.variance_target, svd_solver="full").fit(X_scaled)
return pca.transform(X_scaled), pca, scaler

```

Risk scoring model. A gradient boosting regressor/classifier (LightGBM) predicts forward realized volatility or a discretized risk tier from the PCA-reduced features, tracked with MLflow autologging.

```

# ml/training/phase1_train_risk_model.py
import mlflow
import lightgbm as lgb
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import mean_absolute_error

def train_risk_model(X, y, params: dict):
    mlflow.set_experiment("sentinel-ai/phase1-core-engine")
    with mlflow.start_run(run_name="gbm-risk-scorer"):
        mlflow.log_params(params)
        tscv = TimeSeriesSplit(n_splits=5)
        maes = []
        for train_idx, val_idx in tscv.split(X):
            model = lgb.LGBMRegressor(**params)
            model.fit(X[train_idx], y[train_idx])
            preds = model.predict(X[val_idx])
            maes.append(mean_absolute_error(y[val_idx], preds))
        mlflow.log_metric("cv_mae_mean", sum(maes) / len(maes))
        mlflow.lightgbm.log_model(model, artifact_path="risk_model")
    return model

```

Service exposure. The trained model is served behind services/risk-engine as a FastAPI endpoint (POST /v1/risk-score) that loads the current production model version directly from the MLflow Model Registry at startup, with a scheduled reload on new promotions.

6.2 5.2 Phase 2 — Multimodal Engine (Text + Vision Fusion)

Goal: enrich the Phase 1 tabular signal with a news-sentiment embedding (FinBERT) and a chart-pattern embedding (CNN), fused into a single risk representation.

Text branch. nlp-engine fine-tunes/uses a pretrained financial BERT variant to produce a sentiment logit and a pooled [CLS] embedding per news article associated with an asset/time window; these are aggregated (mean-pooled) across the window.

```

# services/nlp-engine/app/sentiment_model.py
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

class FinancialSentimentEncoder:

```

```

def __init__(self, model_name: str = "ProsusAI/finbert", device: str = "cpu"):
    self.tokenizer = AutoTokenizer.from_pretrained(model_name)
    self.model = AutoModelForSequenceClassification.from_pretrained(model_name).to(
    self.device = device

@torch.no_grad()
def encode(self, texts: list[str]) -> torch.Tensor:
    batch = self.tokenizer(texts, padding=True, truncation=True, return_tensors="pt")
    outputs = self.model(**batch, output_hidden_states=True)
    cls_embeddings = outputs.hidden_states[-1][:, 0, :] # [CLS] token
    return cls_embeddings.mean(dim=0) # window-level pooled embedding

```

Vision branch. vision-engine renders a rolling OHLCV window as a candlestick chart image and passes it through a compact CNN trained to recognize recurring technical patterns.

```

# services/vision-engine/app/pattern_cnn.py
import torch
import torch.nn as nn

class ChartPatternCNN(nn.Module):
    def __init__(self, num_patterns: int = 8, embed_dim: int = 128):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=5, padding=2), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(), nn.AdaptiveAvgPool2d(1)
        )
        self.embed_head = nn.Linear(128, embed_dim)
        self.classifier_head = nn.Linear(embed_dim, num_patterns)

    def forward(self, x: torch.Tensor):
        feats = self.features(x).flatten(1)
        embedding = self.embed_head(feats)
        logits = self.classifier_head(embedding)
        return embedding, logits

```

Fusion. A late-fusion head concatenates the PCA-reduced tabular vector, the pooled text embedding, and the CNN embedding, then passes them through a small feed-forward network to produce the final multimodal risk score:

$$\hat{y} = f_{\theta}([z_{\text{tabular}} \parallel e_{\text{text}} \parallel e_{\text{vision}}])$$

```

# ml/training/phase2_fusion_model.py
import torch.nn as nn
import torch

class MultimodalRiskFusion(nn.Module):

```

```

def __init__(self, tab_dim: int, text_dim: int, vision_dim: int, hidden: int = 256):
    super().__init__()
    in_dim = tab_dim + text_dim + vision_dim
    self.net = nn.Sequential(
        nn.Linear(in_dim, hidden), nn.ReLU(), nn.Dropout(0.2),
        nn.Linear(hidden, hidden // 2), nn.ReLU(),
        nn.Linear(hidden // 2, 1),
    )

def forward(self, z_tab: torch.Tensor, e_text: torch.Tensor, e_vision: torch.Tensor):
    fused = torch.cat([z_tab, e_text, e_vision], dim=-1)
    return self.net(fused)

```

6.3 5.3 Phase 3 — RAG + Grounded LLM Explanation

Goal: ground natural-language explanations of risk scores in retrieved SEC filings, financial news, and economic reports, without allowing the LLM to originate predictions.

Ingestion & indexing. Documents are chunked (e.g., 512-token windows with overlap), embedded with a sentence-embedding model, and stored in a vector index alongside metadata (source, date, ticker).

```

# services/rag-engine/app/indexer.py
from sentence_transformers import SentenceTransformer
import numpy as np

class DocumentIndexer:
    def __init__(self, model_name: str = "sentence-transformers/all-MiniLM-L6-v2"):
        self.encoder = SentenceTransformer(model_name)

    def chunk(self, text: str, chunk_size: int = 512, overlap: int = 64) -> list[str]:
        words = text.split()
        chunks, i = [], 0
        while i < len(words):
            chunks.append(" ".join(words[i:i + chunk_size]))
            i += chunk_size - overlap
        return chunks

    def embed(self, chunks: list[str]) -> np.ndarray:
        return self.encoder.encode(chunks, normalize_embeddings=True)

```

Grounded generation. The explanation prompt is deliberately structured to prevent the LLM from acting as a predictor: it receives the risk engine's already-computed score, the retrieved evidence, and an explicit instruction boundary.

```

# services/rag-engine/app/explainer.py
EXPLANATION_SYSTEM_PROMPT = """You are Sentinel-AI's explanation module.
You are given: (1) a risk score already computed by the risk engine,

```

(2) retrieved source excerpts. Your ONLY task is to explain, in plain language, why the retrieved evidence is consistent (or inconsistent) with the given score. You must NEVER produce a new prediction, price target, buy/sell signal, or personalized financial advice. If the evidence is insufficient, say so explicitly."""

```
def build_explanation_prompt(score: float, evidence_chunks: list[dict]) -> str:
    evidence_block = "\n\n".join(
        f"[Source: {c['source']}, {c['date']}] \n {c['text']}" for c in evidence_chunks
    )
    return f"""Risk score computed by the system: {score:.3f}
```

Retrieved evidence:
{evidence_block}

Explain this score using only the evidence above."""

Retrieval. Top- k retrieval uses cosine similarity between the query embedding q and each indexed chunk embedding d_i :

$$\text{sim}(q, d_i) = \frac{q \cdot d_i}{\|q\| \|d_i\|}$$

6.4 5.4 Phase 4 — Reinforcement Learning Agent

Goal: simulate a closed market environment and progress from tabular Q-Learning to PPO for risk-adjusted, simulation-only portfolio management.

Environment definition. State s_t combines recent returns, the current risk score from Phase 1-2, and current portfolio weights. The action a_t is a (discretized, then continuous) reallocation across assets and cash. The reward is a risk-adjusted return, e.g., a rolling Sharpe-style signal:

$$R_t = \frac{r_{p,t} - r_f}{\sigma_{p,t} + \epsilon}$$

where $r_{p,t}$ is portfolio return, r_f a risk-free rate, and $\sigma_{p,t}$ rolling portfolio volatility.

```
# services/r1-engine/env/market_env.py
```

```
import gymnasium as gym
import numpy as np
from gymnasium import spaces
```

```
class SimulatedMarketEnv(gym.Env):
```

```
    """Closed-loop, simulation-only market environment. No live execution."""
```

```
    def __init__(self, price_data: np.ndarray, risk_scores: np.ndarray, n_assets: int):
```

```

super().__init__()
self.prices = price_data
self.risk_scores = risk_scores
self.n_assets = n_assets
self.action_space = spaces.Box(low=0.0, high=1.0, shape=(n_assets + 1,), dtype=
obs_dim = n_assets * 2 + 1 # returns + risk scores + cash weight
self.observation_space = spaces.Box(low=-np.inf, high=np.inf, shape=(obs_dim,),
self.t = 0
self.weights = np.zeros(n_assets + 1)
self.weights[-1] = 1.0 # start fully in cash

def step(self, action: np.ndarray):
    action = action / (action.sum() + 1e-8) # project onto simplex
    returns = self.prices[self.t + 1] / self.prices[self.t] - 1.0
    portfolio_return = np.dot(action[:-1], returns)
    reward = portfolio_return / (np.std(returns) + 1e-6) # risk-adjusted reward
    self.weights = action
    self.t += 1
    terminated = self.t >= len(self.prices) - 1
    obs = np.concatenate([returns, self.risk_scores[self.t], [action[-1]]])
    return obs.astype(np.float32), float(reward), terminated, False, {}

def reset(self, seed=None, options=None):
    self.t = 0
    self.weights = np.zeros(self.n_assets + 1)
    self.weights[-1] = 1.0
    obs = np.concatenate([np.zeros(self.n_assets), self.risk_scores[0], [1.0]])
    return obs.astype(np.float32), {}

```

Baseline: tabular Q-Learning on a discretized state/action space validates the environment and reward design before investing in a deep RL agent.

Target algorithm: PPO. PPO optimizes a clipped surrogate objective that constrains policy updates to remain close to the previous policy, improving training stability:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

where $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\hat{A}_t$ is the advantage estimate (e.g., via GAE).

```

# ml/training/phase4_train_ppo.py
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv
import mlflow

def train_ppo_agent(env_fn, total_timesteps: int = 200_000):
    mlflow.set_experiment("sentinel-ai/phase4-rl-agent")
    vec_env = DummyVecEnv([env_fn])

```

```

with mlflow.start_run(run_name="ppo-portfolio-agent"):
    model = PPO("MlpPolicy", vec_env, verbose=0, n_steps=2048, batch_size=64, gamma=0.99)
    model.learn(total_timesteps=total_timesteps)
    mlflow.log_param("total_timesteps", total_timesteps)
    model.save("ml/models/ppo_portfolio_agent")
return model

```

6.5 5.5 Phase 5 — Trust, Fairness, and MLOps Maturity

Explainability. SHAP values attribute the GBM/fusion model’s prediction to individual features via Shapley-value decomposition; LIME provides local surrogate explanations for individual predictions.

```

# services/explainability/app/shap_explainer.py
import shap

def explain_prediction(model, X_background: "np.ndarray", X_instance: "np.ndarray"):
    explainer = shap.TreeExplainer(model, X_background)
    shap_values = explainer.shap_values(X_instance)
    return {
        "shap_values": shap_values.tolist(),
        "base_value": float(explainer.expected_value),
    }

```

Adversarial robustness. Small, bounded perturbations δ (e.g., via FGSM: $\delta = \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(x, y))$) are applied to inputs to measure prediction stability under worst-case noise.

Differential privacy. Training with DP-SGD bounds the influence of any single training example by clipping per-example gradients and adding calibrated Gaussian noise, providing an (ϵ, δ) -DP guarantee.

Subgroup bias auditing. For each sensitive/segment attribute (defined only on synthetic population data, never real personal data), the system compares risk-score distributions and error rates across subgroups using metrics such as demographic parity difference and equalized odds difference.

Drift detection. A population stability index (PSI) or Kolmogorov-Smirnov test compares the live feature distribution against the training distribution on a rolling schedule:

$$\text{PSI} = \sum_i (p_i - q_i) \ln \left(\frac{p_i}{q_i} \right)$$

where p_i, q_i are the proportions of observations in bin i for the current and reference distributions, respectively. A PSI above a configured threshold (e.g., 0.2) triggers a retraining pipeline run.

CI/CD. Each service has its own GitHub Actions workflow (matrix-built from the monorepo) that runs lint, unit tests, a Docker build, and — on merge to main — a Kubernetes rolling deploy.

```
# .github/workflows/risk-engine-ci.yml
name: risk-engine-ci
on:
  push:
    paths: ["services/risk-engine/**"]
jobs:
  build-test-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -r services/risk-engine/requirements.txt
      - run: pytest services/risk-engine/tests --cov
      - name: Build Docker image
        run: docker build -t sentinel-ai/risk-engine:${{ github.sha }} services/risk-en
      - name: Deploy to Kubernetes
        if: github.ref == 'refs/heads/main'
        run: kubectl set image deployment/risk-engine risk-engine=sentinel-ai/risk-engi
```

7 Chapter 6: Evaluation Methodology

Because Sentinel-AI is delivered incrementally, each phase defines its own acceptance criteria rather than deferring all evaluation to the end of the project.

Phase	Primary Metric(s)	Acceptance Target
1 — Core Engine	Cross-validated MAE / RMSE of volatility prediction; PCA explained variance	Beats a naive rolling-volatility baseline; PCA retains $\geq 95\%$ variance in $\leq$ half the original feature count
2 — Multimodal	Fusion-model MAE/AUC vs. tabular-only baseline	Statistically significant improvement (e.g., paired t-test, $p < 0.05$) over Phase 1 model on held-out data
3 — RAG/LLM	Groundedness rate (fraction of explanation claims traceable to retrieved evidence); retrieval precision@k	$\geq 90\%$ of explanation sentences traceable to a cited chunk; no ungrounded numeric claims
4 — RL Agent	Cumulative risk-adjusted reward vs. Q-Learning baseline and a buy-and-hold baseline, in simulation only	PPO agent's simulated Sharpe-style reward exceeds both baselines over held-out episodes
5 — Trust/MLOps	SHAP attribution stability, adversarial accuracy drop, DP epsilon budget, subgroup parity gap, drift PSI	Adversarial accuracy drop bounded within a defined tolerance; subgroup parity gap below threshold; drift monitor correctly flags injected distribution shifts in synthetic tests

All metrics are logged to MLflow per run and surfaced on the monitoring dashboard, so that phase acceptance is auditable rather than anecdotal.

8 Chapter 7: Testing Strategy

- **Unit tests** cover feature engineering, PCA transforms, model wrappers, the fusion head, the RAG prompt builder, and the RL environment's step/reset contract, for every service in services/.
- **Integration tests** validate each service's FastAPI contract (request/response schemas, error handling) using `httpx/pytest-asyncio` against a live test instance.

- **Contract tests** at the API Gateway confirm that no route can be constructed which reaches a live-execution capability, enforcing the “simulation-only” boundary at the routing layer, not just by convention.
- **Model tests** assert monotonic sanity properties where applicable (e.g., risk score should not decrease when volatility features increase, holding other inputs fixed).
- **RAG groundedness tests** run a held-out set of (score, evidence) pairs and check that generated explanations do not introduce claims absent from the retrieved evidence.
- **Chaos/robustness tests** in Phase 5 inject adversarial perturbations and synthetic drift to confirm the monitoring stack correctly raises alerts.

9 Chapter 8: Ethical, Safety, and Compliance Considerations

Sentinel-AI’s most important design constraint is not technical but behavioral: **it must never be usable as a real trading or advisory system.** This is enforced at three layers:

1. **Data layer:** only historical (already public, non-real-time) or synthetic data is ingested; no live brokerage feeds or order-routing credentials are ever accepted by the schema.
2. **Service layer:** the rl-engine and risk-engine operate exclusively inside the simulated environment; no service holds a broker API client.
3. **LLM/RAG layer:** the explanation prompt structurally forbids new predictions or advice, and outputs are filtered for advisory language patterns before being returned.

Beyond execution safety, the platform’s Phase 5 trust layer treats fairness and privacy as first-class deliverables rather than an afterthought: subgroup bias auditing and differential privacy experiments are run on every model promoted to the registry, and drift monitoring ensures that a model’s behavior is continuously re-validated against the assumptions it was trained under.

10 Chapter 9: Risk Register (Engineering Delivery Risks)

Risk	Likelihood	Impact	Mitigation
Multimodal fusion underperforms tabular-only baseline	Medium	Medium	Ablation studies per modality before committing to fusion architecture; keep tabular-only model as a documented fallback

Risk	Likelihood	Impact	Mitigation
RAG explanations drift into advisory-sounding language	Medium	High	Prompt-level constraints + a post-generation classifier filter + human-reviewed evaluation set
PPO training instability / reward hacking in simulation	Medium	Medium	Start from validated Q-Learning baseline; reward shaping review; extensive logging of episode trajectories
Scope creep across five phases	High	Medium	Hard phase-gate acceptance criteria (Chapter 6) before starting the next phase
Kubernetes/MLOps overhead slows iteration speed early on	Medium	Low	Use docker-compose for local Phase 1-2 development; introduce Kubernetes manifests starting Phase 3-4 once the service boundary is stable

11 Chapter 10: Future Work

- Extend the RL agent to multi-agent simulation (competing simulated strategies) to study emergent market dynamics.
- Explore a diffusion-based synthetic market data generator to stress-test the risk engine beyond historical regimes.
- Add a formal model card and automated regulatory-style documentation generator, extending the Phase 5 trust layer.
- Investigate on-device / edge-deployable distilled versions of the fusion model for lower-latency inference.

12 Chapter 11: Conclusion

Sentinel-AI is designed as a disciplined, end-to-end demonstration of modern AI engineering applied to a single, sufficiently rich domain — financial risk simulation — under hard safety constraints against real-world use. Its five-phase roadmap intentionally sequences complexity: a solid tabular foundation, multimodal enrichment, grounded explanation, sequential decision-making, and finally a full trust/MLOps layer, each phase gated by explicit, measurable acceptance criteria. The resulting monorepo architecture is deliberately production-shaped — decoupled services, tracked experiments, containerized deployment — so that the finished platform is not only a learning exercise but a credible, portfolio-grade artifact of full-stack AI system design.

13 References

1. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD.
2. Araci, D. (2019). *FinBERT: Financial Sentiment Analysis with Pre-trained Language Models*. arXiv:1908.10063.
3. Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.
4. Schulman, J., et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv:1707.06347.
5. Lundberg, S., & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions (SHAP)*. NeurIPS.
6. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). *“Why Should I Trust You?”: Explaining the Predictions of Any Classifier (LIME)*. KDD.
7. Abadi, M., et al. (2016). *Deep Learning with Differential Privacy*. CCS.
8. Zenodo/MLflow Project. (2024). *MLflow: An Open Source Platform for the Machine Learning Lifecycle*. Documentation.

14 Appendix A: Glossary

- **GBM** — Gradient Boosting Machine, an ensemble of decision trees trained sequentially to correct prior errors.
- **PCA** — Principal Component Analysis, a linear dimensionality-reduction technique.
- **RAG** — Retrieval-Augmented Generation, grounding LLM output in retrieved documents.
- **PPO** — Proximal Policy Optimization, a stable policy-gradient RL algorithm.
- **SHAP** — Shapley Additive Explanations, a game-theoretic feature-attribution method.
- **PSI** — Population Stability Index, a drift-detection statistic.
- **DP-SGD** — Differentially Private Stochastic Gradient Descent.